DEV SANSKRITI VISHWAVIDYALAYA, HARIDWAR

Department of Computer Science

LABORATORY RECORD & VIVA PREPARATION MANUAL

Node.js Practical Lab Record

Comprehensive Practical Exercises (Lab 01 to Lab 08) & Technical Viva Guide

Student Name:	Bhaskar Mall
Enrollment / Roll No.:	23145005
Course:	Bachelor of Computer Applications (BCA)
Semester:	VII Semester
Subject Code & Title:	CS403NOD — Node.js
Academic Session:	2026 – 2027

This practical record covers the design, implementation, and code explanation of Node.js laboratory assignments (Lab 01 through Lab 08), including HTTP server creation, route parsing, asynchronous paradigms (Callbacks, Promises, Async/Await), file operations, EventEmitter systems, and monolithic server integration.

Practical Lab Index

S.No.	Experiment Title / Objectives	Modules / APIs Used	Remarks
1	**Lab 01:** Setup, JavaScript Data Types & Base HTTP Server	`http`, variables, typeof	Completed
2	**Lab 02:** Custom Route Handling & HTTP Status Codes	`http`, URL routing, JSON	Completed
3	**Lab 03 & 04:** Dynamic REST API, Query Params & Modularization	`url`, query parser, local modules	Completed
4	**Lab 05:** Asynchronous Programming (Callbacks, Promises, Async/Await)	`Promise`, `Promise.all`	Completed
5	**Lab 06:** File System Operations (fs) & CLI Notes App	`fs`, `fs.promises`, `process.argv`	Completed
6	**Lab 07:** Event-Driven Architecture with EventEmitter	`events`, Pub/Sub pattern	Completed
7	**Lab 08:** Integrated Monolithic Server & Request Logger	`child_process`, `util`, logging	Completed

1. Core Node.js Architecture & Execution Model

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside a web browser. Understanding its architecture is essential for explaining how it manages concurrent operations on a single main thread.

Question 1: What is Node.js, and what are its two core internal dependencies?

Answer: Node.js is a **JavaScript runtime environment** built on Google Chrome's **V8 JavaScript Engine** and written in C++. It allows JavaScript to interact directly with the operating system, file system, and network sockets. The two foundational dependencies are:

- **Google V8 Engine:** An open-source JavaScript execution engine that compiles JavaScript source code directly into native machine code (Just-In-Time compilation).
- **libuv (C library):** A cross-platform C library that handles the **Event Loop**, non-blocking asynchronous I/O polling, and a background **Thread Pool** (default 4 worker threads) for blocking operations like file system I/O, DNS lookup, and cryptographic computations.

Question 2: How does Node.js handle thousands of concurrent requests if JavaScript is single-threaded?

Answer: Node.js executes application JavaScript on a **single main thread**, but it achieves concurrency using an **Event-Driven, Non-Blocking I/O model** supported by the underlying OS kernel (such as `epoll` on Linux, `IOCP` on Windows) and `libuv`.
When an I/O request (such as a database query or file read) is initiated, Node.js delegates that operation to the OS kernel or libuv thread pool along with a callback function. The main thread is immediately freed to process subsequent incoming requests. Once the operation completes, its callback is queued in the Event Loop and executed when the main call stack is empty.

Question 3: What are the phases of the Node.js Event Loop in order of execution?

Answer: The Event Loop processes events through six primary phases in every tick:

1. **Timers:** Executes callbacks scheduled by `setTimeout()` and `setInterval()` whose timer threshold has expired.
2. **Pending / I/O Callbacks:** Executes deferred system-level callbacks (e.g., specific TCP connection errors).
3. **Idle, Prepare:** Internal libuv bookkeeping routines.
4. **Poll:** Retrieves new I/O events, executes I/O related callbacks, and blocks waiting for events when the queue is empty.
5. **Check:** Executes callbacks invoked by `setImmediate()`.
6. **Close Callbacks:** Executes cleanup handlers, such as `socket.on('close', ...)`.

Note on Microtasks: Callbacks scheduled with `process.nextTick()` run immediately after the current operation finishes on the call stack, before the Event Loop advances to the next phase, followed by Promise microtasks.

Question 4: What are the key differences between Browser JavaScript and Node.js?

Feature	Browser Environment	Node.js Environment
Global Scope	`window`, `document`	`global`, `process`
DOM & UI	Direct access to DOM tree and alerts	No DOM; runs on the server
File System Access	Restricted by sandbox security	Full direct access via `fs` and `path`
Module System	ES Modules (`import/export`)	CommonJS (`require/module.exports`) & ESM

2. Lab Experiments & Code Logic Breakdown

Experiment 01: Setup, Variables, Data Types & Base HTTP Server

Files: `Lab-01/app.js`, `Lab-01/server.js`

Objective: Verify the Node.js installation, demonstrate JavaScript data types using `typeof`, and create an HTTP server using Node's native `http` module without third-party frameworks.

Question: Explain the code inside Lab-01's server.js and why res.end() is required.

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Node.js Lab 01!');
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Explanation:

1. `require('http')` imports the built-in HTTP module provided by the Node.js runtime.
2. `http.createServer()` returns an HTTP server instance. The callback function accepts two stream objects: `req` (IncomingMessage) and `res` (ServerResponse).
3. `res.writeHead(200, { 'Content-Type': 'text/plain' })` sends the HTTP status code (200 OK) and response headers before the body is transmitted.
4. **Why `res.end()` is mandatory:** `res` is a writable stream. Calling `res.end()` signals to the receiving client that all headers and body chunks have been completely transmitted. If `res.end()` is omitted, the client request remains open until a network timeout occurs.

Experiment 02: HTTP Web Server & Custom Route Handling

Files: `Lab-02/server.js`

Objective: Construct an HTTP server capable of serving multiple URL routes (e.g., `/`, `/students`, `/about`) and returning appropriate HTTP status codes and JSON payloads.

Question: How is routing implemented without Express in native Node.js?

Answer: Native Node.js inspects `req.url` and `req.method` on each incoming request using conditional statements:

```
const server = http.createServer((req, res) => {
  const path = req.url.toLowerCase();

  if (path === '/' || path === '/home') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Welcome to Lab 02</h1>');
  } else if (path === '/students') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify([{ id: 1, name: 'Bhaskar', course: 'BCA' }]));
  } else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Route Not Found', statusCode: 404 }));
  }
});
```

Note: Unmatched routes must return status code **404 (Not Found)** to avoid leaving the client in an indeterminate state.

Experiment 03 & 04: Dynamic REST API, URL Query Processing & Modules

Files: `Lab-03/student-server.js`, `Lab-04/advanced-server.js`

Objective: Parse URL query strings, filter datasets based on query parameters, and organize reusable application logic into custom local modules.

Question: How did you extract and sanitize URL query parameters in Lab 04?

Answer: The built-in `url` module parses the incoming request URL. Passing `true` parses the query string into a structured JavaScript object:

```
const url = require('url');
const parsedUrl = url.parse(req.url, true);
const query = parsedUrl.query; // Example: { course: 'BCA', minMarks: '70' }
const pathname = parsedUrl.pathname;

if (pathname === '/students') {
  let results = [...studentDatabase];

  if (query.course) {
    results = results.filter(s => s.course.toLowerCase() === query.course.toLowerCase());
  }

  if (query.minMarks) {
    const min = parseFloat(query.minMarks);
    if (!isNaN(min)) {
      results = results.filter(s => s.marks >= min);
    }
  }

  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ total: results.length, data: results }));
}
```

Question: What is the difference between Core Modules, Local Modules, and Third-Party Modules?

Answer:

- **Core Modules:** Built directly into the Node.js binary (e.g., `http`, `fs`, `path`, `events`, `os`). Loaded directly by name: `require('fs')`. No npm installation required.
- **Local Modules:** Custom JavaScript files written in the project (e.g., `./modules/logger.js`). Loaded via relative paths: `require('./modules/logger')`.
- **Third-Party Modules:** Packages downloaded from the npm registry (e.g., `slugify`). Stored in `node_modules/` and imported by package name: `require('slugify')`.

Experiment 05: Asynchronous Paradigms: Callbacks, Promises & Async/Await

Files: `Lab-05/callback-version.js`, `Lab-05/async-await-version.js`

Objective: Compare asynchronous patterns in JavaScript using a multi-step food delivery simulation (Place Order → Cook → Dispatch → Deliver).

Question: Compare Callbacks, Promises, and Async/Await. What is Callback Hell?

Answer:

- **Callbacks:** Functions passed as arguments to execute upon task completion. Chaining multiple asynchronous operations leads to deeply nested code known as *Callback Hell*, which makes error handling cumbersome.
- **Promises:** Objects representing the eventual completion or failure of an asynchronous action. They have three states: *Pending*, *Fulfilled*, and *Rejected*. They resolve nested structures through `.then()` and `.catch()`.
- **Async/Await:** Syntactic sugar introduced in ES2017 built on top of Promises. Allows asynchronous code to be written cleanly with standard `try...catch` blocks.

```
async function processDeliveryOrder(orderId) {
  try {
    const order = await placeOrder(orderId);
    const food = await prepareFood(order);
    const rider = await dispatchRider(food);
    const delivered = await deliverFood(rider);
    console.log('Order completed:', delivered);
  } catch (err) {
    console.error('Error during delivery:', err.message);
  }
}
```

Question: What is Promise.all() and when is it preferred over sequential awaits?

Answer: `Promise.all([p1, p2, p3])` executes an array of promises concurrently. It resolves when all promises complete, or rejects immediately if any single promise fails.
Usage: When tasks are independent (such as fetching data from three separate APIs or reading three unrelated files), sequential `await` takes the total combined time, whereas `Promise.all()` runs them in parallel, completing in approximately the time of the slowest operation.

Experiment 06: File System (fs) Module & CLI Notes Application

Files: `Lab-06/fs-demo.js`, `Lab-06/add-note.js`, `Lab-06/read-notes.js`

Objective: Perform file read and write operations using synchronous and asynchronous methods, and parse command-line arguments using `process.argv`.

> **Question: Why does fs.readFile() not block the program while fs.readFileSync() does?**
>
> **Answer:**
> `fs.readFile()` is asynchronous. When called, Node.js delegates the disk read request to the `libuv` thread pool and continues running subsequent JavaScript code without pausing. The callback is invoked only when the file reading finishes.
> In contrast, `fs.readFileSync()` is synchronous. It blocks the main JavaScript thread completely until the entire file has been read from storage into memory. No other requests can be handled while this operation is in progress.

> **Question: In the Notes App, what happens if two users execute add-note.js simultaneously?**
>
> **Answer:**
> A **Race Condition** occurs. When two separate processes append data to the same file simultaneously without a locking mechanism, write operations can overlap, resulting in garbled text or lost updates. In concurrent production applications, transactional databases (like PostgreSQL or MongoDB) or message queues are used instead of shared flat files.

> **Question: How do you read command-line arguments in Node.js?**
>
> **Answer:** Via the global `process.argv` array:
> `process.argv[0]` = Path to the Node.js executable (`node.exe`).
> `process.argv[1]` = Path to the JavaScript script file being executed.
> `process.argv[2]` onwards = User-supplied command-line parameters.
> Example: In `node add-note.js "Study Node"`, `process.argv[2]` equals `"Study Node"`.

Experiment 07: EventEmitter Architecture & Event-Driven Programming

Files: `Lab-07/emitter.js`, `Lab-07/07-practical-example/app.js`

Objective: Implement publisher-subscriber event workflows using the built-in `EventEmitter` class from the `events` module.

Question: Explain EventEmitter methods: on, emit, once, removeListener.

Answer:

- `emitter.on(eventName, listener)` : Registers a listener callback function that executes every time the event is emitted.
- `emitter.emit(eventName, ...args)` : Triggers the event synchronously, invoking all registered listeners in order.
- `emitter.once(eventName, listener)` : Registers a one-time listener that is automatically removed after its first invocation.
- `emitter.removeListener(eventName, listener)` : Unregisters a specific listener function to avoid memory leaks.

```
const EventEmitter = require('events');
class OrderTracker extends EventEmitter {}
const tracker = new OrderTracker();

tracker.on('orderPlaced', (data) => console.log('Notification sent:', data.customer));
tracker.once('orderDelivered', (data) => console.log('Delivered successfully'));

tracker.emit('orderPlaced', { orderId: 101, customer: 'Bhaskar' });
```

Important: If an `'error'` event is emitted without an active listener, Node.js throws an uncaught exception and terminates the process. Therefore, `emitter.on('error', handler)` should always be registered.

Experiment 08: Integrated Monolithic Lab Server & System Logger

Files: `Lab-08/server.js`, `Lab-08/labs.js`, `Lab-08/modules/logger.js`

Objective: Consolidate previous lab implementations into an integrated server without rewriting the original code, using module delegation and child process execution.

Question: How did you integrate all previous labs without rewriting them?

Answer:

1. **Server Labs (Lab 01, Lab 02):** The master server imports the handler function using `require(labPath)`. For incoming requests matching `/labs/:id/app/...`, it adjusts the URL prefix and delegates directly to the existing handler: `handler(req, res)`.
2. **Script Labs (Lab 03 to Lab 07):** The server executes the script file using Node's built-in `child_process.execFile()` wrapped with `util.promisify`:

```
const { execFile } = require('child_process');
const { promisify } = require('util');
const execFileAsync = promisify(execFile);

const result = await execFileAsync(process.execPath, [labFilePath], { timeout: 5000 });
res.end(JSON.stringify({ ok: true, output: result.stdout }));
```

Question: How does the custom EventEmitter logging system in Lab 08 work?

Answer: Inside `modules/logger.js`, a custom class extending `EventEmitter` is used. For each incoming request, the server emits a log event: `logger.emit("request", req.method, req.url);`
The listener formats an ISO timestamp and asynchronously appends the entry to `logs/server.log`:

```
logger.on("request", (method, requestUrl) => {
  const entry = new Date().toISOString() + " " + method + " " + requestUrl;
  console.log(entry);
  fs.appendFile(LOG_FILE, entry + "\n", (err) => {
    if (err) logger.emit("error", err);
  });
});
```

Question: What is the difference between execFile, exec, spawn, and fork?

Answer:

- `execFile`: Runs an executable file directly without invoking a system shell. It is secure against shell injection and has lower overhead.
- `exec`: Spawns a system shell (e.g., cmd or bash) to execute a command string. It buffers output but carries security risks if inputs are unsanitized.
- `spawn`: Spawns a new process and streams data via `stdout` and `stderr`, suitable for handling large outputs without buffer overflow.
- `fork`: A specialized version of `spawn` designed specifically to launch new Node.js instances with built-in Inter-Process Communication (IPC).

3. Essential Conceptual Viva Questions & Answers

Question 1: What is the difference between require() and import?

`require()` is synchronous and belongs to the CommonJS module system traditionally used in Node.js. `import` is asynchronous, statically analyzed, and belongs to the ES Module standard.

Question 2: What is the difference between module.exports and exports?

`module.exports` is the actual object exported by the module when required. `exports` is simply a reference pointing to `module.exports`. Reassigning `exports = {}` breaks the reference, so assignments should be made directly to `module.exports`.

Question 3: What is the purpose of package.json and package-lock.json?

`package.json` contains project metadata, scripts, and semantic version ranges. `package-lock.json` stores the exact dependency tree with specific versions and hashes to ensure identical installations across environments.

Question 4: What do caret (^) and tilde (~) mean in dependency versioning?

`^1.6.0` allows updates that do not modify the major version (e.g., 1.6.1 or 1.7.0). `~1.6.0` allows only patch-level updates (e.g., 1.6.1, but not 1.7.0).

Question 5: What is the difference between process.nextTick() and setImmediate()?

`process.nextTick()` runs immediately after the current operation finishes on the call stack, before the Event Loop advances. `setImmediate()` runs during the Check phase of the Event Loop.

Question 6: What is a Stream in Node.js and what are its types?

A stream is an interface for reading or writing data chunk-by-chunk without loading the entire payload into RAM. The four types are: **Readable** (e.g., `fs.createReadStream`), **Writable** (e.g., `res`, `fs.createWriteStream`), **Duplex** (e.g., TCP socket), and **Transform** (e.g., `zlib` compression).

Question 7: What is an error-first callback?

A Node.js standard where the first parameter of a callback is reserved for an error object (if an error occurred), while subsequent parameters carry the result: `(err, result) => { if (err) { ... } }`.

Question 8: What is a Buffer in Node.js?

A global class used to handle raw binary memory directly outside V8's garbage-collected heap, commonly used when dealing with TCP network packets and disk file streams.

Question 9: What causes the EADDRINUSE error and how is it resolved?

It occurs when a server attempts to listen on a TCP port that is already in use by another running process. It is resolved by terminating the existing process (e.g., via `taskkill /F /IM node.exe` on Windows) or by specifying a different port.

Question 10: What is the difference between path.join() and path.resolve()?

`path.join()` concatenates path segments using the operating system's separator (`\` on Windows, `/` on Linux). `path.resolve()` resolves segments into an absolute path based on the current working directory.

Question 11: Why is blocking the Event Loop dangerous in Node.js?

Because Node.js executes JavaScript on a single thread. If an intensive synchronous CPU calculation or synchronous file read blocks that thread, all other connected clients are blocked from receiving responses until the operation finishes.

Question 12: What is util.promisify?

A built-in utility in the `util` module that converts standard error-first callback functions into Promise-returning functions, allowing them to be used with `await`.

4. Practical Execution Reference & Run Commands

Experiment	Directory	Terminal Command	Verification URL / Output
Lab 01	`Lab-01/`	`node server.js`	`http://localhost:3000/`
Lab 02	`Lab-02/`	`node server.js`	`http://localhost:3000/students`
Lab 03	`Lab-03/`	`node student-server.js`	`http://localhost:3000/students?id=1`
Lab 04	`Lab-04/`	`node advanced-server.js`	`http://localhost:3000/students?course=BCA&minMarks=70`
Lab 05	`Lab-05/`	`node async-await-version.js`	Console logs showing step-by-step food delivery timestamps

Lab 06	`Lab-06/`	`node fs-demo.js`
		`node add-note.js "Prepare Viva"`
		`node read-notes.js`
			Checks notes created in `notes.txt` on disk

Lab 07	`Lab-07/`	`node emitter.js`	Console output showing single and multi-listener events
Lab 08	`Lab-08/`	`node server.js`	`http://localhost:3000/` & `http://localhost:3000/health`
Portfolio	`NodeJS-lab/`	`node portfolio-server.js`	`http://localhost:5050/`

Verification Note — Lab 08 Execution:
1. Navigate to directory: `cd Lab-08`
2. Start server: `node server.js`
3. Visit `http://localhost:3000/health` → confirms server status and active environment.
4. Visit `http://localhost:3000/labs/02/app/students?course=BCA` → verifies route delegation to Lab 02.
5. Visit `http://localhost:3000/labs/05/run` → executes script lab via `child_process.execFile`.